[1]:
```python
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import os
import pickle
import random
import seaborn as sns
import cv2
import splitfolders
```

[2]:
```python
df = pd.read_csv("/Users/clementarhin/Python/jupyter_notes/data/MachineLearning/gender_classification_v7.csv")
```

[11]:
```python
df = df.drop_duplicates()
```

[12]:
```python
df.describe()
```

[12]:

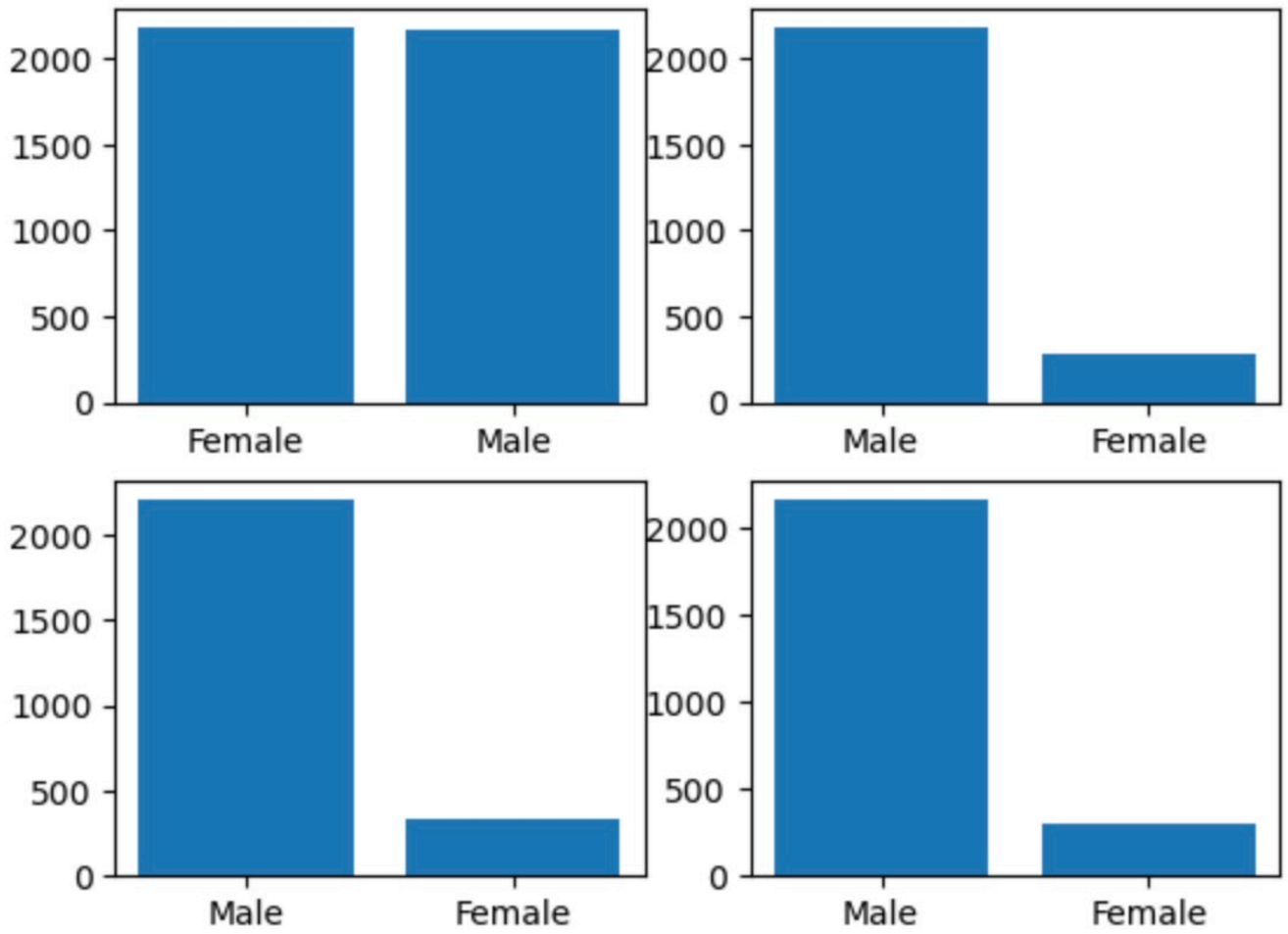
	long_hair	forehead_width_cm	forehead_height_cm	nose_wide	nose_long	lips_thin	distance_nose_to_lip_long
count	3233.000000	3233.000000	3233.000000	3233.000000	3233.000000	3233.000000	3233.000000
mean	0.822456	13.217198	5.966038	0.536344	0.557686	0.538509	0.545314
std	0.382188	1.115993	0.549789	0.498755	0.496738	0.498592	0.498019
min	0.000000	11.400000	5.100000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	12.300000	5.500000	0.000000	0.000000	0.000000	0.000000
50%	1.000000	13.200000	5.900000	1.000000	1.000000	1.000000	1.000000
75%	1.000000	14.100000	6.400000	1.000000	1.000000	1.000000	1.000000
max	1.000000	15.500000	7.100000	1.000000	1.000000	1.000000	1.000000

[4]:
```python
df.shape
```

[4]: (5001, 8)

[72]:
```
gender_long_hair = df[df['long_hair'] == 1]
gender_nose_wide = df[df['nose_wide'] == 1]
gender_nose_long = df[df['nose_long'] ==1]
gender_lips_thin = df[df['lips_thin'] ==1]
gender_distance_nose_to_lip_long = df[ df['distance_nose_to_lip_long'] ==1]
gender_counts = df['gender'].value_counts()
```

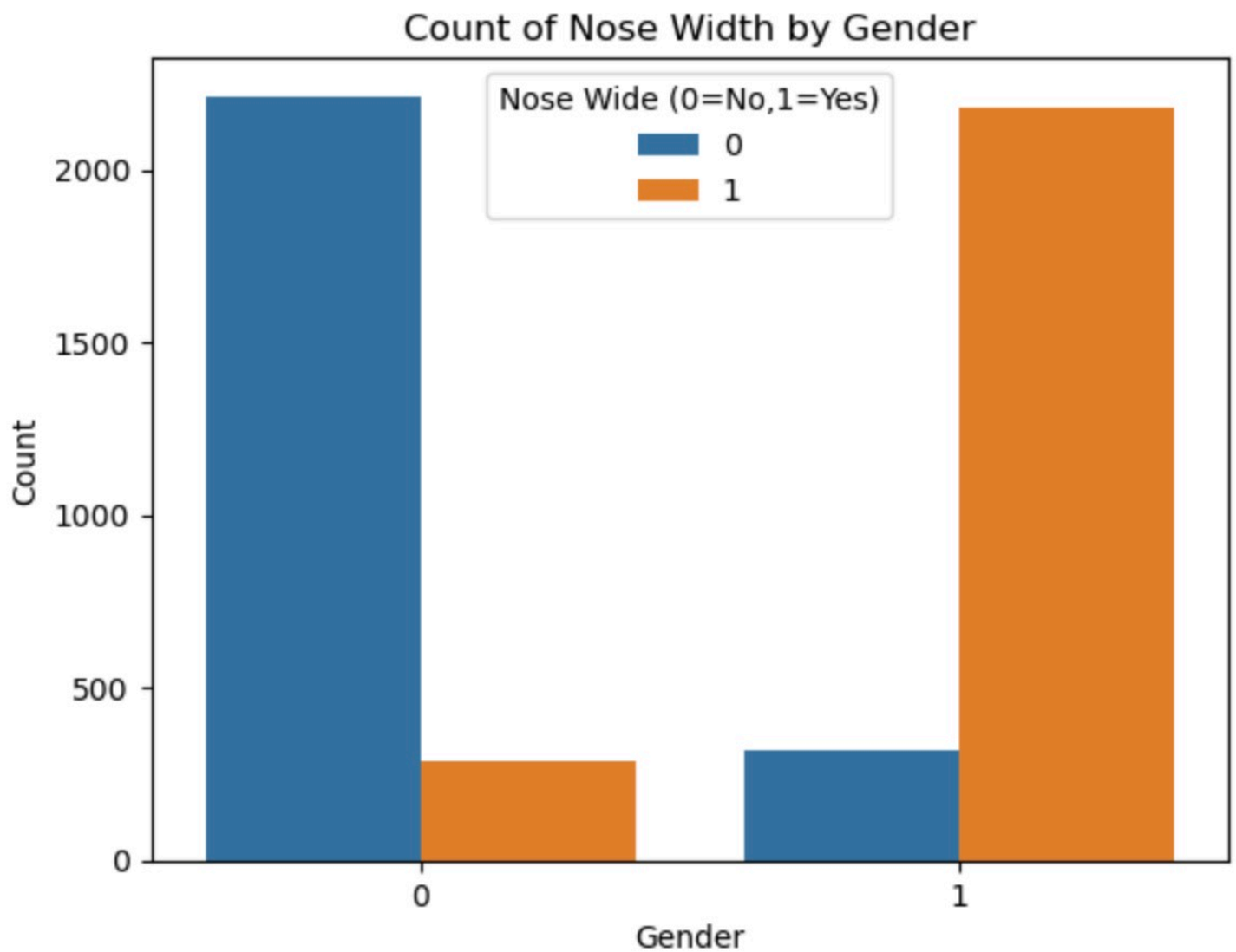
[73]:
```
fig,((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2)
ax1.bar(gender_long_hair['gender'].value_counts().index, gender_long_hair['gender'].value_counts())
ax2.bar(gender_nose_wide['gender'].value_counts().index,gender_nose_wide['gender'].value_counts())
ax3.bar(gender_nose_long['gender'].value_counts().index, gender_nose_long['gender'].value_counts())
ax4.bar(gender_lips_thin['gender'].value_counts().index, gender_lips_thin['gender'].value_counts())
plt.show()
```



[77]:
```
df['gender'] = df['gender'].apply(lambda x:1 if x == 'Male' else 0)
```

[78]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='gender', hue='nose_wide', data=df)
plt.title("Count of Nose Width by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.legend(title='Nose Wide (0=No,1=Yes)')
plt.show()
```



[79]:
```
counts = df.groupby(['gender', 'nose_wide']).size().reset_index(name='count')
print(counts)
```

[84]:
```python
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

[85]:
```python
X = df.drop('gender', axis=1)
y = df['gender']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=40)
```

[86]:
```python
model = RandomForestClassifier()
model.fit(X_train, y_train)
```

[86]:
```
▾ RandomForestClassifier
RandomForestClassifier()
```

[87]:
```python
y_preds = model.predict(X_test)
```

[88]:
```python
accuracy = accuracy_score(y_preds, y_test)
accuracy
```

[88]:
```
0.957042957042957
```

[89]:
```python
importances = model.feature_importances_
feature_names = X.columns

for name, importance in zip(feature_names, importances):
    print(f"{name}: {importance:.4f}")
```

```
long_hair: 0.0029
forehead_width_cm: 0.0886
forehead_height_cm: 0.0583
nose_wide: 0.2541
nose_long: 0.1853
lips_thin: 0.1653
distance_nose_to_lip_long: 0.2455
```

[30]:
```python
from sklearn import metrics
new_pred = model.predict(X_test)

cm = metrics.confusion_matrix(y_test, new_pred)
print("Confusion matrix:\n", cm)

if cm.shape == (2, 2):
    tn, fp, fn, tp = cm.ravel()
    print(f"TN: {tn}, FP: {fp}, FN: {fn}, TP: {tp}")

print("Accuracy:", metrics.accuracy_score(y_test, new_pred))
print("Precision (weighted):", metrics.precision_score(y_test, new_pred, average='weighted'))
print("Recall (weighted):", metrics.recall_score(y_test, new_pred, average='weighted'))
print("F1-Score (weighted):", metrics.f1_score(y_test, new_pred, average='weighted'))

print("\nClassification Report:")
print(metrics.classification_report(y_test, new_pred))
```

```
Confusion matrix:
 [[264  13]
 [ 17 353]]
TN: 264, FP: 13, FN: 17, TP: 353
Accuracy: 0.9536321483771252
Precision (weighted): 0.9537865786396136
Recall (weighted): 0.9536321483771252
F1-Score (weighted): 0.9536723417968934

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.95      0.95       277
           1       0.96      0.95      0.96       370

    accuracy                           0.95       647
   macro avg       0.95      0.95      0.95       647
weighted avg       0.95      0.95      0.95       647
```

[ ]: